

INFORME TÉCNICO

Sistema de Gestión Hospitalaria

Diseño, Modelado y Programación de Base de Datos

Fecha de emisión:	Abril 2026
Versión del documento:	1.0
Estado:	Final
Elaborado por:	Equipo de Desarrollo — Leslie, Cris y Sergio, Lluvia
Base de datos:	PostgreSQL 18.3 — bd_marin

Introducción

La presente información, documenta el proceso de análisis, diseño y desarrollo de la base de datos relacional para el Sistema de Gestión Hospitalaria. El proyecto partió de cuatro casos de uso planteados por el área de requerimientos, los cuales describían necesidades reales de un entorno hospitalario: la gestión de tipos sanguíneos y compatibilidades de transfusión, la clasificación de áreas del hospital y sus aforos, el registro y seguimiento de enfermedades por temporada, y la categorización de tratamientos y medicamentos según costo y duración.

El equipo de trabajo estuvo conformado por tres colaboradores con roles bien definidos: Leslie se encargó del análisis y la planeación del diagrama entidad-relación; Cris asumió la responsabilidad de la programación y creación de los objetos de base de datos; y Sergio se dedicó a la reorganización y normalización de las relaciones entre tablas, asegurando la integridad referencial del modelo final.

Equipo de Trabajo y Roles

A continuación, se describen las responsabilidades concretas de cada integrante del equipo durante el desarrollo del proyecto:

Leslie — Planeación y Diagramación

Leslie fue responsable de traducir los casos de uso en un modelo conceptual claro. Su trabajo consistió en identificar las entidades principales del sistema (pacientes, personal, áreas del hospital, tipos sanguíneos, enfermedades, tratamientos y medicamentos), definir sus atributos y esbozar las relaciones entre ellas mediante un diagrama entidad-relación (DER).

Leslie también se encargó de establecer las reglas de negocio que luego se plasmaron como restricciones en la base de datos, como los niveles de urgencia de las áreas, los umbrales de costo y la clasificación de los tipos de tratamiento.

Cris — Programación e Implementación

Cris tomó el modelo conceptual aprobado y lo implementó integralmente en PostgreSQL. Sus tareas incluyeron la creación de todas las tablas con sus tipos de dato, restricciones CHECK, secuencias y valores por defecto. Adicionalmente, desarrolló las funciones y triggers necesarios para automatizar validaciones críticas del negocio: verificar que solo doctores puedan recibir citas, que solo enfermeros puedan realizar ventas de medicamentos, que el stock en inventario sea suficiente antes de registrar una venta, que la medicina requiera receta cuando corresponda, y que el aforo máximo de cada área del hospital no sea superado al registrar un ingreso.

Sergio — Reorganización de Relaciones

Sergio tuvo a su cargo la revisión y normalización del esquema relacional. Verificó que todas las llaves foráneas estuvieran correctamente definidas y que las dependencias entre tablas respetaran las formas normales. Reorganizó las referencias entre entidades como compatibilidad sanguínea (donante-receptor), historial tratamientos (paciente-doctor-tratamiento), detalle receta (receta-medicina) y detalle venta (venta-inventario-medicina), garantizando que el modelo no presentara redundancias ni inconsistencias. También revisó que las restricciones de unicidad fueran coherentes con las reglas del negocio.

Análisis de Casos de Uso

Caso 1 — Tipos Sanguíneos, Compatibilidad de Transfusiones y Presión Arterial

El primer caso de uso planteaba la necesidad de registrar los ocho grupos sanguíneos existentes (A+, A-, B+, B-, AB+, AB-, O+, O-) junto con sus reglas de compatibilidad para donaciones y recepciones de sangre, así como su propensión diferenciada a niveles de presión arterial alta o baja.

Solución implementada en la base de datos:

Se crearon dos tablas para dar respuesta a este caso. La tabla `tiposanguineos` almacena cada grupo con su nombre, descripción y el atributo `propensiondpresion`, el cual puede tomar los valores 'Hipertension', 'Hipotension' o 'Neutral', reflejando las diferencias clínicas documentadas en el caso (por ejemplo, el grupo B+ muestra mayor susceptibilidad a la hipertensión, mientras que A- y AB- presentan tendencia a la hipotensión).

La tabla `compatibilidadesanguinea` resuelve la complejidad de las relaciones de donación y recepción mediante una estructura de tabla de cruce que referencia dos veces a `tiposanguineos`: una como donante y otra como receptor. Esta tabla permite registrar todos los pares compatibles descritos en el caso (por ejemplo, O- como donante universal puede donar a los ocho grupos, mientras que AB+ como receptor universal puede recibir de todos). Se añadió una restricción UNIQUE sobre el par (`donanteid`, `receptorid`) para evitar duplicados, y el campo `esvalido` permite marcar compatibilidades vigentes o anuladas.

Caso 2 — Tipos de Pacientes, Áreas Hospitalarias y Niveles de Urgencia

El segundo caso de uso describía la estructura interna de un hospital estándar, compuesta por cinco tipos de estancias o áreas de atención: urgencias, piso, quirófano, terapia intensiva y salas de tratamiento especial. Cada área tiene un nivel de urgencia (alta, media o baja) y un aforo máximo de camillas.

Solución implementada en la base de datos:

Se diseñó la tabla `areashospital`, que contiene los atributos `nombre`, `nivelurgencia` y `aforomaximo`. El campo `nivelurgencia` está controlado mediante una restricción `CHECK` que solo permite los valores 'Alta', 'Media' o 'Baja', en correspondencia con los niveles descritos en el caso. El campo `aforomaximo` incluye una restricción `CHECK` que garantiza que el valor sea siempre mayor a cero.

Adicionalmente, la tabla `ingresoshospitalarios` registra cada estancia de un paciente en un área, con su estado (En Curso, Alta o Defuncion). Para respetar los límites de aforo, Cris implementó el trigger `fn_ingresos_validaraforo`, que antes de cada inserción de ingreso consulta cuántas camas están actualmente ocupadas en el área de destino y lanza una excepción si el aforo máximo ya fue alcanzado, garantizando así la integridad operativa.

Caso 3 — Aglomeración de Enfermedades por Temporadas

El tercer caso de uso planteaba la necesidad de registrar enfermedades cuya incidencia varía por temporadas o por eventos excepcionales, identificando su origen (pandemia, causa climatológica, plaga a la fauna, plaga a la flora o endémica) y la forma de atención correspondiente a cada tipo de brote.

Solución implementada en la base de datos:

Se creó la tabla `catalogoenfermedades`, que almacena el nombre de cada enfermedad, su descripción, el campo `origenbrote` (controlado por `CHECK` con los valores 'Pandemia', 'Climatologica', 'Plaga Fauna', 'Plaga Flora', 'Endemica' y 'Otra') y un campo `esendemica` de tipo booleano que indica si la enfermedad persiste de forma permanente en la naturaleza, tal como se especificaba en el caso.

Las consultas médicas se registran en la tabla `consultas`, que referencia tanto la cita médica como la enfermedad diagnosticada, permitiendo con ello construir estadísticas de incidencia por enfermedad, área del hospital y período de tiempo. Esto facilita la identificación de temporadas de alta demanda y la planificación de insumos como antibióticos, antihistamínicos o vacunas.

Caso 4 — Costos y Tipos de Tratamientos y Medicamentos

El cuarto caso de uso planteaba la necesidad de clasificar tratamientos y medicamentos según su costo (extremo, alto, medio o bajo) y su duración (crónica, temporal o inmediata), diferenciando además entre medicamento genérico y de patente, y entre tratamiento privado y popular.

Solución implementada en la base de datos:

Se implementaron dos tablas complementarias. La tabla tratamientos clasifica cada tratamiento con los atributos umbralcosto (CHECK: 'Extremo', 'Alto', 'Medio', 'Bajo'), duracion (CHECK: 'Cronica', 'Temporal', 'Inmediata') y tipoatencion (CHECK: 'Privado', 'Popular'), cubriendo de forma completa la taxonomía descrita en el caso. El campo costo almacena el valor numérico exacto del tratamiento.

Por su parte, la tabla medicinas incorpora los atributos tipomedicamento (CHECK: 'Generico', 'Patente') y umbralcosto (CHECK: 'Extremo', 'Alto', 'Medio', 'Bajo'), permitiendo filtrar y comparar medicamentos dentro de cada categoría de costo. La tabla inventariomedicinas complementa este módulo al registrar las existencias disponibles en farmacia, y los triggers fn_detventa_validarreceta y fn_detventa_descontarinventario automatizan la validación de receta y el descuento de stock en cada venta.

Diagramado de la estructura de los usos de la base de datos.

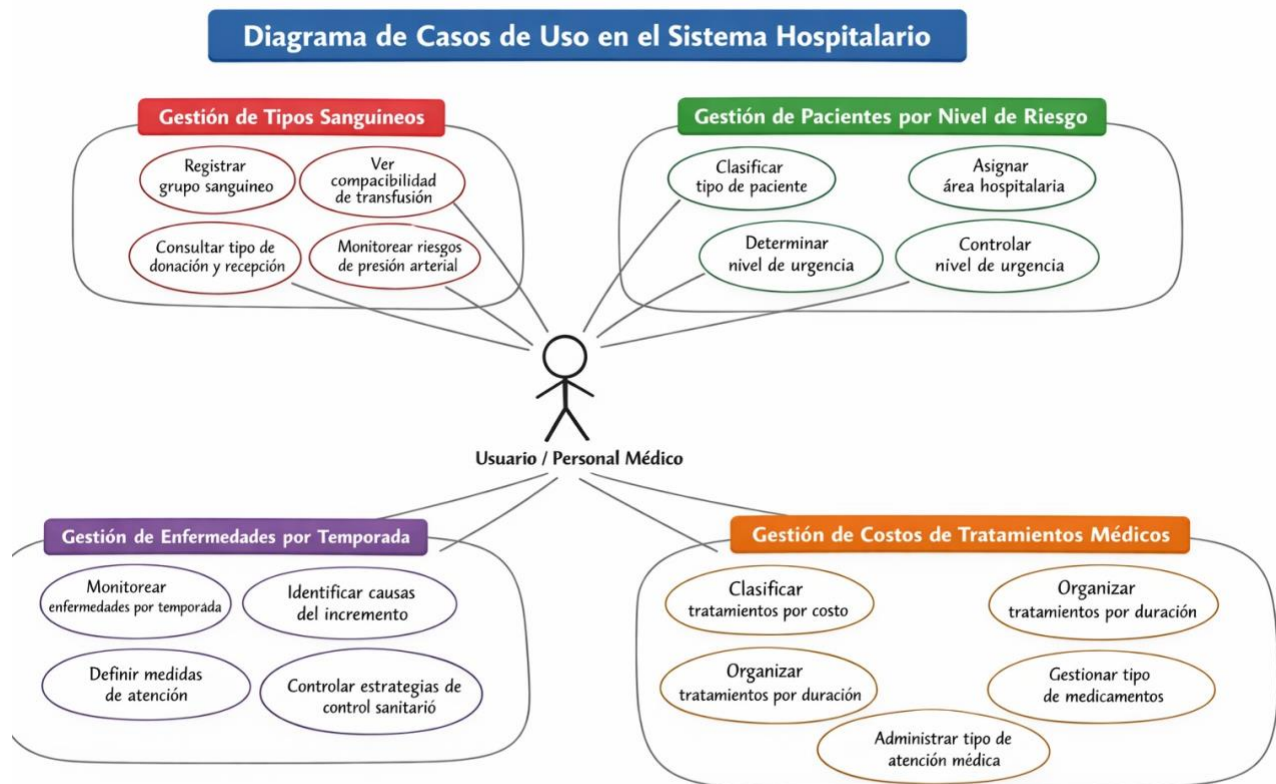


Diagrama realizado por la integrante Lluvia Hdz Flores.

Estructura General de la Base de Datos

El modelo físico resultante en PostgreSQL está compuesto por las siguientes tablas principales, organizadas según los módulos funcionales del sistema:

Tabla	Módulo / Caso	Descripción breve
tiposanguineos	Caso 1 — Tipos sanguíneos	Grupos sanguíneos y propensión a presión
compatibilidadesanguinea	Caso 1 — Transfusiones	Tabla cruce donante-receptor
areashospital	Caso 2 — Áreas y aforos	Estancias, urgencia y capacidad máxima
ingresoshospitalarios	Caso 2 — Ingresos	Registro de estadías por paciente y área
catalogoenfermedades	Caso 3 — Enfermedades	Catálogo con origen de brote y endemicidad
consultas	Caso 3 — Diagnóstico	Vincula citas con enfermedades diagnosticadas
tratamientos	Caso 4 — Tratamientos	Costo, duración y tipo de atención
medicinas	Caso 4 — Medicamentos	Tipo (genérico/patente) y umbral de costo
inventariomedicinas	Caso 4 — Inventario	Stock disponible por medicina
pacientes	Transversal	Datos de pacientes con tipo sanguíneo
personal	Transversal	Doctores, enfermeros y administrativos
citas	Transversal	Agendamiento doctor-paciente
recetas / detallereceta	Transversal	Recetas médicas y sus medicamentos
ventasmedicina / detalleventa	Transversal	Ventas en farmacia con control de stock
historialtratamientos	Transversal	Historial de tratamientos por paciente

Triggers y Funciones de Validación

Una parte fundamental del trabajo de Cris fue el desarrollo de funciones trigger que automatizan las reglas de negocio directamente en la capa de base de datos, reduciendo la dependencia de validaciones en la capa de aplicación. Los triggers implementados son:

- `fn_citas_validardoctor`: Verifica antes de insertar una cita que el personal asignado como doctor tenga efectivamente el rol 'Doctor'. Rechaza la inserción si se intenta asignar a un enfermero o administrativo.
- `fn_venta_validarenfermero`: Valida que el personal registrado como responsable de una venta de medicamentos sea de tipo 'Enfermero', garantizando que solo personal autorizado realice despacho en farmacia.
- `fn_detventa_descontarinventario`: Al registrar el detalle de una venta, verifica que exista stock suficiente en inventario y realiza el descuento automático de la cantidad vendida.
- `fn_detventa_validarreceta`: Comprueba que si un medicamento requiere receta (campo `RequiereReceta = TRUE`), la venta correspondiente tenga una receta médica asociada antes de proceder.
- `fn_ingresos_validaraforo`: Antes de registrar un nuevo ingreso hospitalario, cuenta las camas actualmente ocupadas en el área de destino y bloquea el ingreso si se alcanzó el aforo máximo definido en `areashospital`.